# Day 114: Naive Bayes From Scratch — Real Text Classification

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 3 | Complete study guide

Watch: Naive Bayes, Clearly Explained!!! (StatQuest) | Understanding Multinomial and Bernoulli Naive Bayes: A Step-by-Step Example (AI Wizardry)

Script: `day114_naive_bayes.py` · every number in this guide is copied by program from `day114_run_output.txt`, the real output of that script.

Naive Bayes written from scratch and run on 67,349 real movie-review phrases (SST-2). It matches scikit-learn to 1.4e-13. On the full training set it scores 0.8085 against 0.8165 for Day 98's logistic regression; but with little data the order reverses, and Naive Bayes leads on all 5 subsets up to 3,000 items, as Ng and Jordan (2002) predicted. Its 'naive' assumption shows up in its confidence: 0.4484 of its answers claim 99% certainty, against 0.0688 for logistic regression, and writing every sentence twice exactly doubles its evidence.

## 0. Where today fits

**DEFINITION — start here, no prior knowledge assumed**

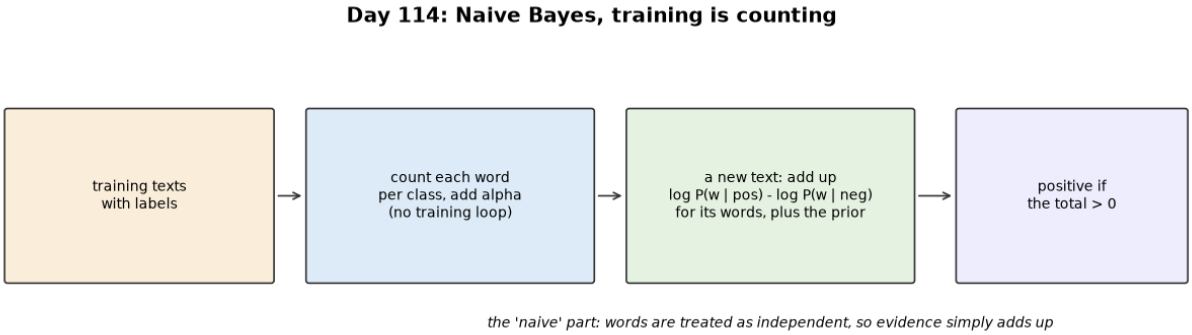
**Naive Bayes** applies Bayes' rule: P(class | text) is proportional to P(class) × P(text | class). The 'naive' part assumes the words are independent given the class, so P(text | class) is simply the product of each word's probability in that class.

Training is **counting**: how often each word appears in each class, plus a small **smoothing** count $\alpha$ so that an unseen word does not make a probability zero.

**REAL-WORLD SCENARIO**

Sorting post by keywords: 'invoice' and 'payment' suggest bills, 'sale' and 'offer' suggest adverts. Each word adds a little evidence, and nobody stops to ask whether 'invoice' and 'payment' tend to come together.

**SYSTEM DIAGRAM**




*Naive Bayes: training is counting, and prediction adds up one log ratio per word.*

### Today's part for the week's build

`day114_exercise.py` is the three kinds of Naive Bayes in one class, with Day 98's tokeniser. Its real output:

**ACTUAL OUTPUT**

```
'good fun movie': by hand 0.008 / (0.008 + 0.002) = 0.8; multinomial gives 0.8000
with an unseen word, 'good fun movie zebra': 0.8000
'good good movie': multinomial log-odds 1.3863 (good counted twice); binary 0.6931 (once)
Bernoulli, empty text (only absent words speak): log-odds 0.2877; 'good': 1.3863
bigrams of 'not bad at all': ['not', 'bad', 'at', 'all', 'not bad', 'bad at', 'at all']
```

Data: SST-2 (Week 15) from the local cache: 67,349 training phrases and sentences and 872 validation sentences, used as the test set. Day 98's tokeniser, logistic regression and calibration measure are imported unchanged.

### How to read each section

| Step | What it gives you |
|---|---|
| DEFINITION | What the idea is, in plain words. Start here. |
| REAL-WORLD SCENARIO | An everyday situation where the same idea shows up. |
| MATHEMATICS | The formula, when the section introduces one. |
| SPECIFICATION | The exact setup of the experiment. |
| IMPLEMENTATION | The real code from the script. |
| ACTUAL OUTPUT | What the script really printed, copied unchanged. |
| What it means | How to read the numbers, and what they do and do not show. |
| Why choose this? / Why NOT this? | When the idea is the right tool, and what it costs. |

## 1. By hand: four tiny reviews

**DEFINITION — start here, no prior knowledge assumed**

Two positive reviews ('good movie', 'great fun') and two negative ('bad movie', 'boring bad'). Each class has 4 word slots, the vocabulary has 6 words, and $\alpha = 1$.

**REAL-WORLD SCENARIO**

A tally of how often each word turned up in good reviews and in bad ones.

**MATHEMATICS**

$$P(w \mid c) = (\text{count of } w \text{ in class } c + \alpha) / (\text{all words in class } c + \alpha V)$$

$$\log P(c \mid \text{text}) = \log P(c) + \Sigma_{\text{words}} \log P(w \mid c) + \text{constant}$$

**IMPLEMENTATION**

```
def fit(self, texts, y):
    self.vocab = {}; ids, rows = self.encode(texts, grow=True); V = len(self.vocab); a =
        self.alpha
    self.prior = np.log(np.bincount(y, minlength=2) / len(y))
    N = np.stack([np.bincount(ids[y[rows] == c], minlength=V) for c in (0, 1)]).astype(float)
    if self.kind == "bernoulli":
        n = np.bincount(y, minlength=2)[:, None]
        p = (N + a) / (n + 2 * a)
        self.lp, self.lq = np.log(p), np.log(1 - p)
    else:
        self.lp = np.log((N + a) / (N.sum(1, keepdims=True) + a * V))
    return self
```

**ACTUAL OUTPUT**

```
positive: 'good movie', 'great fun'; negative: 'bad movie', 'boring bad'; 6 distinct words, 4 word slots per
    class, alpha 1
for 'good fun movie': positive (1+1)/(4+6) x (1+1)/(4+6) x (1+1)/(4+6) = 0.008; negative (0+1)/10 x (0+1)/10 x
    (1+1)/10 = 0.002
so P(positive) = 0.008 / (0.008 + 0.002) = 0.8; the class gives 0.8000
an unseen word changes nothing: 'good fun movie zebra' -> 0.8000
Actual output from this lesson's run.
```

### What it means

The class reproduces the hand calculation, 0.8000. An unseen word ('zebra') changes nothing, because it is in neither class's counts.

**WHY CHOOSE THIS?**

Every probability can be checked with a calculator.

**WHY NOT THIS? (TRADEOFFS)**

Four documents cannot show what goes wrong with real language.

## 2. Correctness

**DEFINITION — start here, no prior knowledge assumed**

scikit-learn's MultinomialNB, given the same words through the same tokeniser, must produce the same log-odds.

**REAL-WORLD SCENARIO**

Two accountants totting up the same receipts should reach the same total.

**ACTUAL OUTPUT**

```
vocabulary: 13,816 words here, 13,816 in scikit-learn
log-odds on the 872 validation sentences: largest gap 1.4e-13; the same predictions on 1.0000
Actual output from this lesson's run.
```

### What it means

Both find 13,816 words, and the log-odds agree to 1.4e-13, rounding only. Every prediction matches.

**WHY CHOOSE THIS?**

An exact check: the model is pure counting, so nothing should differ.

**WHY NOT THIS? (TRADEOFFS)**

Agreement shows the formula is implemented right, not that the model is good.

## 3. The three kinds, against logistic regression and DistilBERT

**DEFINITION — start here, no prior knowledge assumed**

**Multinomial** counts every occurrence. **Binary** counts each word once per text. **Bernoulli** also scores the words that are absent. Adding **bigrams** (adjacent word pairs) gives the model 'not good' as a unit.

**REAL-WORLD SCENARIO**

Judging a review by how often it says 'great', by whether it says 'great' at all, or also by which words it never uses.

**MATHEMATICS**

$$\textit{Bernoulli: } \log P(\text{text} \mid c) = \Sigma_{w \text{ present}} \log p_{w,c} + \Sigma_{w \text{ absent}} \log(1 - p_{w,c})$$

**ACTUAL OUTPUT**

```
model                              accuracy        95% interval
multinomial NB                       0.8073      [0.7821, 0.8326]
binary NB                            0.8085      [0.7821, 0.8337]
Bernoulli NB                         0.8062      [0.7810, 0.8326]
binary NB, + bigrams                 0.8165      [0.7913, 0.8429]
logistic regression (Day 98)         0.8165      [0.7913, 0.8417]
DistilBERT (Day 98, its output)      0.9106
binary NB and logistic regression give the same answer on 0.8704 of sentences
Actual output from this lesson's run.
```

### What it means

The three kinds are within noise of each other (0.8062 to 0.8085). Bigrams lift binary Naive Bayes to 0.8165, level with logistic regression, whose 0.8165 reproduces Day 98's number exactly. DistilBERT, pretrained on far more text, is about 9 points ahead (0.9106). The two counting models agree on only 0.8704 of sentences, so they make different mistakes.

**WHY CHOOSE THIS?**

Seconds to train on 67,000 texts, with no tuning loop at all.

**WHY NOT THIS? (TRADEOFFS)**

About 9 points behind a pretrained Transformer on this task.

## 4. Learning curves: who wins with little data?

**DEFINITION — start here, no prior knowledge assumed**

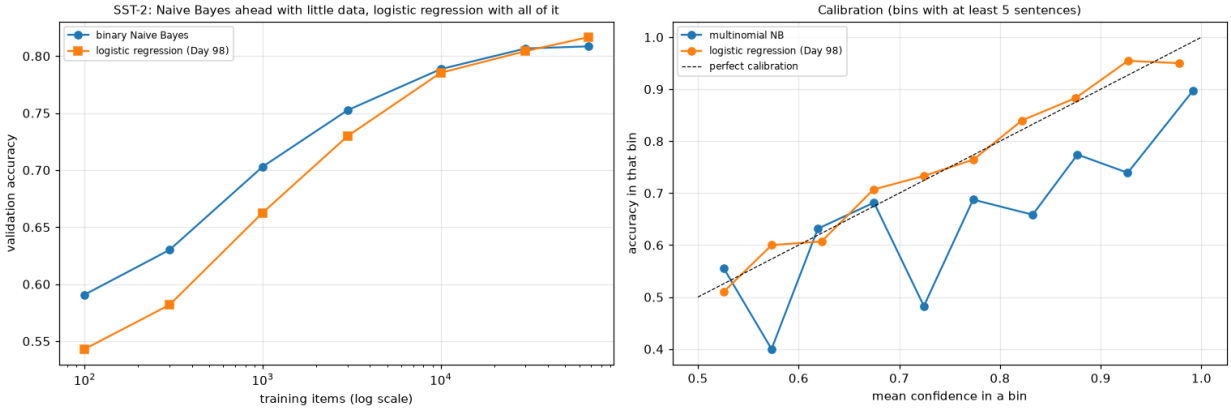
Ng and Jordan (2002) showed that a generative model like Naive Bayes approaches its best accuracy with fewer examples than a discriminative one like logistic regression, which usually ends higher. Each size here uses 5 random subsets.

**REAL-WORLD SCENARIO**

A rough rule of thumb is useful from the first day; a carefully tuned process pays off only after much more experience.

**ACTUAL OUTPUT**

```
 training items   binary NB   logistic   NB - LR  NB ahead in
            100      0.5908     0.5431   +0.0477        5 of 5
            300      0.6300     0.5819   +0.0482        5 of 5
          1,000      0.7030     0.6626   +0.0404        5 of 5
          3,000      0.7525     0.7298   +0.0227        5 of 5
         10,000      0.7885     0.7853   +0.0032        4 of 5
         30,000      0.8067     0.8044   +0.0023        2 of 5
         67,349      0.8085     0.8165   -0.0080        0 of 1
logistic regression is Day 98's recipe, unchanged (300 Adam steps, words in at least 2 training
    texts); with few items its settings were not re-tuned
Actual output from this lesson's run.
```




*Left: validation accuracy against training items. Right: calibration, accuracy against confidence in bins of at least 5 sentences.*

### What it means

The textbook result, on real data. Up to 3,000 items Naive Bayes is ahead on every subset (by +0.0477 with 100 items); the gap closes by 10,000 and 30,000, and with all 67,349 logistic regression is ahead (-0.0080). One caution: Day 98's logistic regression was not re-tuned for small samples, so part of its early deficit may be its settings.

**WHY CHOOSE THIS?**

With a few hundred labelled texts, Naive Bayes is a strong first model.

**WHY NOT THIS? (TRADEOFFS)**

Its lead disappears as data grows; it is rarely the final model.

## 5. Smoothing

**DEFINITION — start here, no prior knowledge assumed**

$\alpha$ is added to every count. Too small, and rare words dominate with extreme ratios; too large, and real differences are flattened. $\alpha$ is chosen by 5-fold cross-validation on the training items; validation is printed alongside, not used.

**REAL-WORLD SCENARIO**

Adding one imaginary vote for every candidate, so that no one starts at exactly zero.

ACTUAL OUTPUT

```
   alpha  CV accuracy  validation
    0.01       0.8795      0.7856
    0.03       0.8796      0.7867
     0.1       0.8796      0.7936
     0.3       0.8785      0.7982
       1       0.8764      0.8085
       3       0.8719      0.8131
      10       0.8572      0.8005
chosen by CV: alpha 0.1 (the validation column is printed for comparison, not used to choose)
Actual output from this lesson's run.
```

### What it means

Cross-validation chooses $\alpha = 0.1$, which scores 0.7936 on validation, while $\alpha = 3$ would have scored 0.8131. The two columns disagree, most likely because the data differ (not tested here): training items are mostly short phrases cut from sentences, validation items are whole sentences, and cross-validation can only judge on phrases. The model is chosen for the data it was shown, which is not quite the data it is tested on.

WHY CHOOSE THIS?

One setting, cheap to choose.

WHY NOT THIS? (TRADEOFFS)

The right value depends on how alike training and test texts are.

## 6. The independence assumption

DEFINITION — start here, no prior knowledge assumed

If words were truly independent, each would add fresh evidence. They are not: 'hilarious' and 'funny' tend to appear together, and the model counts the same evidence twice. The symptom is overconfidence, measured by the **expected calibration error** (ECE, Day 98).

REAL-WORLD SCENARIO

Five friends repeating the same rumour do not make it five times as true.

ACTUAL OUTPUT

```
multinomial NB        : mean confidence 0.9170, accuracy 0.8073, answers at least 99% sure 0.4484, ECE 0.1112
binary NB             : mean confidence 0.9158, accuracy 0.8085, answers at least 99% sure 0.4335, ECE 0.1189
logistic regression   : mean confidence 0.8157, accuracy 0.8165, answers at least 99% sure 0.0688, ECE 0.0202
each validation sentence written twice: multinomial log-odds change by a factor of 2.0000 (median, excluding
    the prior); binary NB by 1.0000
multinomial NB on the doubled sentences: accuracy 0.8096, mean confidence 0.9614, ECE 0.1538
Actual output from this lesson's run.
```

### What it means

Naive Bayes is about 81% accurate and about 92% sure: 0.4484 of its answers claim at least 99% confidence, and its ECE is 0.1112, against 0.0202 for logistic regression. Writing each sentence twice, which adds no information, exactly doubles the multinomial model's evidence (2.0000) and raises its ECE to 0.1538; the binary kind, which counts a word once, is unmoved (1.0000). Its rankings are useful; its probabilities are not to be taken literally.

**WHY CHOOSE THIS?**

The ranking of texts by score is often good enough.

**WHY NOT THIS? (TRADEOFFS)**

Probabilities cannot be trusted without recalibration.

## 7. What it learned, and negation

**DEFINITION — start here, no prior knowledge assumed**

The log ratio log P(w | positive) – log P(w | negative) shows what each word means to the model. Negation is the classic weakness of a bag of words: 'not good' contains 'good'.

**REAL-WORLD SCENARIO**

A phrasebook that translates word by word gets 'not bad' wrong.

**ACTUAL OUTPUT**

```
most positive (words in at least 100 texts): touching +4.36, solid +3.92, perfectly +3.92, unique +3.88,
    terrific +3.52, powerful +3.39, hilarious +3.31, portrait +3.29, inventive +3.18, beautifully +3.16
most negative: unfunny -4.86, poorly -4.79, ugly -4.37, worse -4.24, stupid -3.98, pretentious -3.79, shallow
    -3.71, worst -3.63, mess -3.51, contrived -3.22
  'good': log ratio +0.988
  'not': log ratio -0.676
  'bad': log ratio -2.377
  'funny': log ratio +0.961
validation sentences with a negation: 183; without: 689
  binary NB                    : with negation 0.7322, without 0.8287
  binary NB, + bigrams         : with negation 0.7322, without 0.8389
  logistic regression (Day 98)  : with negation 0.7049, without 0.8462
  bigram 'not bad': log ratio +1.138
  bigram 'not good': log ratio -1.724
  bigram 'not funny': log ratio -2.948
Actual output from this lesson's run.
```

### What it means

The lists read like a film critic's vocabulary: 'touching' +4.36, 'unfunny' -4.86. 'not' itself leans negative (-0.676). Sentences with a negation are harder for every model (0.7322 against 0.8287 for binary Naive Bayes). Bigrams learn the right meanings, 'not bad' is positive (+1.138) and 'not funny' strongly negative (-2.948), yet accuracy on negated sentences does not move (0.7322 with or without them). A likely reason, not tested here, is that many negations in real sentences reach further than the next word.

**WHY CHOOSE THIS?**

Every learned weight is a readable word ratio.

**WHY NOT THIS? (TRADEOFFS)**

Word order beyond pairs is invisible to it.

## Interview preparation — technical questions

**Q: Why is it called naive?**

A: It assumes the words are independent given the class, which is false for language.

**Q: Why use logarithms?**

A: A product of thousands of small probabilities underflows to zero; a sum of logs does not.

**Q: What is Laplace smoothing for?**

A: Without it, one unseen word makes a class's probability exactly zero.

**Q: Multinomial, binary or Bernoulli?**

A: Here all three were within noise (0.8062 to 0.8085); binary is a common default for short texts.

**Q: How do you check a Naive Bayes implementation?**

A: Against a hand calculation, and against a library: here 1.4e-13.

## Interview preparation — theoretical questions

**Q: When does Naive Bayes beat logistic regression?**

A: With little data: here on every subset up to 3,000 items.

**Q: Why is Naive Bayes overconfident?**

A: Correlated words are counted as separate evidence: 0.4484 of answers claimed 99% certainty.

**Q: Why did cross-validation pick a different alpha from the test set?**

A: Training items are phrases, test items sentences: a distribution shift.

**Q: What can a bag of words not capture?**

A: Word order: negation is the classic case.

## Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Naive Bayes is training by counting. It is exact, instant and readable, and with a few thousand labelled texts it beats logistic regression on real reviews. Its independence assumption makes its probabilities far too confident, and a bag of words cannot see word order, so it is a first model, a baseline, and rarely the last.

- Matches scikit-learn to 1.4e-13.
- All training items: binary NB 0.8085, with bigrams 0.8165; logistic regression 0.8165; DistilBERT 0.9106.
- 100 to 3,000 items: Naive Bayes ahead on every subset.
- ECE 0.1112 against 0.0202; doubled sentences give 2.0000 times the evidence.

### Suggested watch order, if starting from zero

- 1. Naive Bayes, Clearly Explained!!! (StatQuest) — the idea on a spam example.
- 2. Understanding Multinomial and Bernoulli Naive Bayes: A Step-by-Step Example (AI Wizardry) — the multinomial and Bernoulli versions, worked through.

### Day 115 preview

- **k-nearest neighbours** from scratch, with a real demonstration of the curse of dimensionality.